Physics: rigid bodies, SDF collision and destruction

Status: current alpha tree. This manual covers the physics layer a game uses for moving bodies, SDF terrain, destructible objects and the new impact/damage contract. It is an engineering guide, not a promise that the engine owns every game rule. Damage thresholds, loot, win conditions, fracture art and particle effects stay with the game.

0. Backend selection, capabilities and projectiles

The engine has two implementations behind the same `PhysicsSystem` API:

Backend	Best for	Joints	CCD	Determinism	Recommended body count
`PhysicsBackendKind::Box2D`	general rigid bodies, ropes, vehicles, ragdolls, editor	yes	yes	platform-dependent float	up to about 10000
`PhysicsBackendKind::Dod`	large joint-free strategy worlds, simple shapes, projectiles	no	swept segment queries only	fixed-order SoA update	up to `kMaxTrackedPhysicsBodies`
`PhysicsBackendKind::Auto`	default policy	chooses Box2D when joints are required	chooses Box2D when a joint/CCD feature is requested	chooses DOD when deterministic or a body budget of 4096+ is requested	depends on the resolved backend

`Auto` is the project default. Set `PhysicsDesc::backend` before `PhysicsSystem::Initialize`, call `SelectBackend` before the world is live, or use `SetProjectPolicy` to update the policy copied into a Play-In-Editor world. A live world is never swapped in place.

Project setting

The project file stores:

```
[physics]
backend = 0      # 0 Auto, 1 Box2D, 2 DOD
body_budget = 0  # Auto picks DOD at 4096 or more
```

The same values are edited in **Project Settings > Physics**. They apply to the

next Play-In-Editor session or game launch; the editor's reference world keeps its own backend.

Capabilities

`PhysicsSystem::Capabilities()` reports joints, CCD, deterministic mode, large worlds and a recommended body ceiling. Do not infer support from the backend name; query the capabilities. A DOD world asked to create a joint is an explicit error or falls back to Box2D only when the policy is `Auto`.

Projectile modes

`Projectile2D` is an ECS component with these fields:

Field	Meaning
`Mode`	`kProjectileModeAuto`, `Hitscan`, `Swept`, `Dynamic`
`Speed`	pixels per second; zero reads the body velocity
`Radius`	projectile radius used to choose Auto mode
`MinWallThickness`	smallest wall the developer wants to guarantee
`Direction`	normalized travel direction
`MaxSubsteps`	reserved cap for dynamic integration
`CollisionMask`	category mask for the segment query

`Auto` compares `Speed * fixedDt` with `MinWallThickness * 0.5`. Small radii become `Hitscan`; larger or physical projectiles become `Swept`; slow projectiles stay `Dynamic` and receive the Box2D bullet/CCD flag. A developer can always force a mode, so a designer who knows the wall thickness and speed is not at the mercy of the heuristic.

`ProjectileSystem::Step` runs before the next physics step for hitscan/swept projectiles and records a `ProjectileHitEvent` with the projectile, target, point, normal and speed. Game code decides whether to apply damage, spawn an impact or despawn the projectile.

Script and visual language

Text FemboyScript exposes `physics_get_backend()`, `physics_set_backend(backend)` and `physics_get_capabilities()`. The LogicGraph palette has `Physics Get Backend, Physics Set Backend and Physics Capabilities` nodes that compile to the same natives and AOT calls. Existing scripts that never mention a backend

use `Auto`.

Tests and benchmarks

- `physics_backend` covers Auto selection, capability reporting, segment queries

and projectile tunneling.

- `script_physics` covers the text and bytecode bridge.
- `physics_backend_benchmark` reports Box2D and DOD ms/frame for a configurable

body count (default 2000, override on the command line).

1. Units, coordinates and step order

- World positions are pixels. Box2D works in metres; `PhysicsDesc::pixelsPerMeter`

is the only conversion point.

- `Transform2D::Position` is the authoritative gameplay position. `RigidBody2D`

stores the packed Box2D id and the last interpolated pose.

- Simulation runs in `Phase::FixedUpdate` with a fixed `dt`. Rendering

interpolates between `PreviousPosition` and `Position`.

- The recommended order is: input, movement, `PhysicsSystem::Step`,

`SdfCollisionSystem::Step`, damage/impact handling, fracture, then render

interpolation.

- A body that is destroyed between steps must not leave a packed shape id behind;

`PhysicsSystem::DestroyAllBodies` and the registry sweep handle teardown.

2. Rigid bodies and joints

`RigidBody2D` and `Collider2D` are the ECS components. They are plain data and do

not include `box2d.h`. `PhysicsSystem` owns the world; the joint, transport and

wheel systems are explicitly Box2D-aware translation units behind the same

packed handles.

```
Fby::PhysicsSystem Physics;
Fby::PhysicsDesc Desc;
Desc.gravity = Fby::Vector2D{0.0f, 980.0f};
Physics.Initialize(Desc);
```

```
const Fby::Entity Box = Registry.create();
Fby::Transform2D Transform;
Transform.Position = Fby::Vector2D{120.0f, 80.0f};
Registry.emplace<Fby::Transform2D>(Box, Transform);

Fby::RigidBody2D Body;
Body.Type = Fby::BodyType::Dynamic;
Registry.emplace<Fby::RigidBody2D>(Box, Body);

Fby::Collider2D Collider;
Collider.HalfExtents = Fby::Vector2D{16.0f, 16.0f};
Registry.emplace<Fby::Collider2D>(Box, Collider);

Physics.EnsureBodies(Registry);
```

Joints (`PhysicsJoints.hpp`) cover chain, rope and spring connections with a fixed-capacity graph. Cloth and shallow water are separate systems; they do not run through the rigid-body solver.

3. SDF terrain

`SdfField` is a non-owning view of a dense distance grid. Negative values are solid, positive values are outside. `SdfField::Sample` is bilinear and `SdfField::Gradient` returns the outward normal.

`SdfBrickMap` stores the same field as 16x16 bricks: `Empty`, `Full` or `Narrow`. The brick map is the broadphase and the GPU mirror; it is not a second physics representation.

`SdfCollisionSystem` samples a collider's centre and corners, finds the deepest penetration, takes the gradient normal and corrects the body with a velocity and an impulse. The correction is intentionally velocity-based, not a transform teleport, so contact continuity and continuous collision detection survive.

3.1 Contact data

Each resolved contact is a `SdfContact`:

Field	Meaning
`entity`	Body entity
`point`	Deepest contact point in world pixels
`normal`	Outward surface normal
`penetration`	Penetration depth
`impulse`	Correction impulse magnitude

`RelativeSpeed`	Approach speed along the normal before correction
`KineticEnergy`	`0.5 * Mass * RelativeSpeed^2`
`Mass`	Body mass at the moment of contact
`sequence`	Monotonic contact sequence

`RelativeSpeed`, `KineticEnergy` and `Mass` are captured **before** the body is pushed out. A damage rule should use these fields rather than the post-correction push-out speed, otherwise the same collision produces different damage depending on the correction budget.

`SdfCollisionSystem::Contacts()` returns at most `kMaxContacts` contacts per step. `SdfCollisionSystem::Manifolds()` returns up to four points per body for debug and for future solver work.

3.2 Limits of the SDF contact path

- SDF contacts are body-versus-field only. Body-versus-body contacts still come from Box2D; use the Box2D contact velocity and impulse for damage between two dynamic bodies.
- The correction impulse is not a physically exact contact impulse. The new `KineticEnergy` field is the honest input for a damage rule.
- Thin features disappear at the field resolution. A cut thinner than roughly two cells is not a reliable gameplay boundary.
- The gradient is undefined on a medial axis. The collision system falls back to the previous normal instead of inventing a direction.

4. Destructible geometry

`SdfDestructible` is a local dense field owned by one object.

- `Configure` allocates the local grid from `SdfDestructibleDesc`.
- `Subtract(SdfCarveOp)` appends a subtraction operation and rebakes only the affected bricks.
- The operation list is the source of truth. `SaveOps` stores the list and `LoadOps` replays it; the baked samples are a cache.

- `kMaxOps` is 64. A game that needs more detail should use pre-fractured pieces instead of an unbounded carve list.
- `SdfWorld::Imprint` merges a settled object's field into the world. Until that point the world field does not see the object.

`SdfFracture.hpp` splits a destructible into a grid of pieces for prototyping. `SdfDebris.hpp` spawns deterministic box fragments from the material a carve removes. Both are deliberately simple; production destruction should use authored fragments with their own sprites, convex shapes, masses and materials. `SdfDestructionCommands.hpp` records a carve operation in the command log. A replay that receives the same operation list rebuilds the same hole.

5. Impact, material and damage infrastructure

The new header is `Engine/Physics/SdfDamage.hpp`. It is fixed-capacity, deterministic, allocation-free and does not depend on the renderer.

5.1 Material profile

`SdfMaterialProfile` is data, not a behaviour:

Field	Default	Meaning
`Toughness`	1.0	Resistance to damage
`Hardness`	1.0	Damage multiplier for the same impact
`MinKineticEnergy`	0.0	Energy below which the hit is ignored
`EnergyToDamage`	1.0	Damage per unit of effective energy
`ThicknessExponent`	1.0	How strongly local thickness reduces damage
`MaximumDamageFraction`	1.0	Upper bound removed by one impact

`Clamp()` keeps every value in a usable range. The profile is deliberately small: armour layers, hit zones, elemental resistances and scripted invulnerability are game rules and belong in a custom policy or in the game's own material table.

5.2 Parts and connections

A modular object is a `SdfDamageModel` with up to 32 parts and 64 connections.

```
Fby::SdfDamageModel Damage;
```

```
Fby::SdfPart Shell;
Shell.Center = Fby::Vector2D{0.0f, 0.0f};
Shell.Radius = 24.0f;
Shell.Integrity = 1.0f;
Shell.Mass = 4.0f;
Shell.Material = 0;
Damage.AddPart(Shell);

Fby::SdfPart Core;
Core.Center = Fby::Vector2D{0.0f, 0.0f};
Core.Radius = 10.0f;
Core.Integrity = 1.0f;
Core.Mass = 8.0f;
Core.Material = 1;
Damage.AddPart(Core);

Fby::SdfConnection Link;
Link.PartA = 0;
Link.PartB = 1;
Link.Strength = 0.8f;
Damage.AddConnection(Link);
```

`SdfPart::Integrity` is normalized to 0..1. `SdfPart::Radius` is a cheap point-hit proxy; the game can keep the real fragment shape in its own asset.

`SdfConnection` is the structural graph: a broken part breaks every connection that touches it. `ApplyConnectionLoad` accumulates external load and breaks the link when `Load` reaches `Strength`, which gives a game a simple place to hang its own stress propagation.

5.3 Impact

`SdfImpact` is the resolved hit:

Field	Meaning
`Target`	Entity that was hit
`Point`	World contact point
`Normal`	Outward normal
`RelativeSpeed`	Approach speed
`Impulse`	Correction impulse
`KineticEnergy`	Pre-correction impact energy
`Mass`	Body mass
`LocalThickness`	Material depth along the inward normal; zero means unknown and the default rule uses one unit

Use `MakeSdfImpact(contact, field, maxThickness)` to copy a contact and estimate the local thickness. `SdfThicknessInward(field, point, normal, maxThickness)`

marches from the surface into the solid; it returns zero for a point already outside.

```
for (const Fby::SdfContact& Contact : Collision.Contacts())
{
    const Fby::SdfImpact Impact = Fby::MakeSdfImpact(Contact, Field, 24.0f);

    const std::uint32_t PartIndex = Damage.FindPartAtPoint(Impact.Point);
    if (PartIndex == Fby::kSdfInvalidDamagePart)
    {
        continue;
    }

    const Fby::SdfPart* Part = Damage.Part(PartIndex);
    const Fby::SdfMaterialProfile& Material = Materials[Part->Material];
    const Fby::SdfImpactResult Result = Damage.ApplyImpact(Impact, PartIndex,
Material);

    if (Result.PartBroken)
    {
        // Game-owned: spawn authored fragments, debris, audio and VFX.
    }
}
```

`ApplyImpactAtPoint` is a convenience for one material profile:

```
const Fby::SdfImpactResult Result = Damage.ApplyImpactAtPoint(Impact, Material);
```

5.4 Default rule and custom policy

`SdfDefaultDamage` computes:

```
damage = clamp(
    (KineticEnergy - MinKineticEnergy) * EnergyToDamage * Hardness
    / (Toughness * LocalThickness^ThicknessExponent),
    0,
    MaximumDamageFraction)
```

The result is a fraction of the part's integrity. A custom policy replaces the formula:

```
float ArmourPolicy(void* User, const Fby::SdfImpact& Impact,
                   const Fby::SdfPart& Part,
                   const Fby::SdfMaterialProfile& Material) noexcept
{
    const auto* Layers = static_cast<const ArmourLayers*>(User);
    // Game rule: direction, weak point, elemental resistance, invulnerability.
    return ComputeArmourDamage(*Layers, Impact, Part, Material);
}
const Fby::SdfImpactResult Result =
    Damage.ApplyImpact(Impact, PartIndex, Material, &ArmourPolicy, &Layers);
```

A policy must return 0..1, must not be random unless the randomness is part of

the recorded simulation state, and must not read the wall clock.

5.5 Break, fracture and debris handoff

When `Integrity` reaches zero:

- `SdfImpactResult::PartBroken` is true;
- `SdfImpactResult::BrokenConnections` counts the links that changed state;
- `SdfPart::Broken()` and `SdfConnection::Broken()` become true;
- further impacts on that part are ignored.

The engine does not decide what a broken part becomes. The game can:

1. call `FractureGrid` for a prototype;
2. spawn authored fragment entities with `SpawnDebris` or its own asset;
3. remove the object and imprint the carved field with `SdfWorld::Imprint`;
4. play audio, VFX, loot and win/lose logic.

`FractureGrid` copies the complete op list into every piece. That is correct for small prototypes but expensive for a complex structure; authored fragments are the production path.

6. Determinism, persistence and networking

- `SdfDamageModel` has fixed arrays, no RNG and no allocation. The same inputs produce the same state.
- A custom damage policy becomes part of the simulation contract. If a replay cannot run the same policy, the replay must fail loudly instead of diverging.
- `SdfCollisionSystem` contacts are not serialized. Damage is: the game must store part integrity, broken flags and connection flags, or replay the damage commands in the same order.
- `SdfDestructionCommands` already records carve operations. A future damage command should record the impact or the resulting damage, not both.
- Do not feed GPU readback or a renderer-side particle state into damage. The authoritative state stays on the CPU.
- New serialized fields need a format version bump and a migration test. The current `SdfDamage` structs are runtime-only; no `.fsdf` layout changed.

7. What the engine deliberately does not do

- No universal continuum/FEM deformation. It is expensive, hard to tune and usually not what a 2D sprite game needs.
- No automatic texture reprojection onto a carved or fractured shape. Use a damage mask, an interior/cross-section layer, edge decals and an authored fragment atlas.
- No automatic deformation of the collider from a cosmetic sprite warp. A visual dent can use a mesh warp, but the authoritative collider remains a convex shape or the SDF field.
- No universal loot, score, win/lose or oxygen/bubble rule. The game owns those.
- No engine-owned fracture art. The engine provides the damage state and the existing fracture/debris primitives; the art pipeline provides the pieces.

8. Recommended 2D destruction tiers

Tier	Authoring	Runtime	Use
0	One sprite, damaged-state sprites	Swap sprite, decals	Cheap props
1	Authored convex fragments	Break connections, spawn pieces	Default gameplay destruction
2	Sprite mesh with control points	Cosmetic warp, no collider change	Dents and bends
3	Local SDF field	`SdfCarveOp`, imprint	Bullet holes and detail
4	Off-engine solver	Game/plugin	True soft material simulation

Tier 1 is the recommended default. Tier 3 is for local detail, not for the structural graph. Tier 4 should not be promised by a general-purpose 2D engine.

9. Budgets and tests

- `SdfDamageModel`: 32 parts, 64 connections, one pass over the parts for a point lookup, one pass over the connections for a break.
- `SdfCollisionSystem`: one bilinear sample plus four gradient taps per body per fixed step, capped at `kMaxContacts` contacts.
- `SdfDestructible`: one local brick rebake per carve, capped at 64 operations.

- Contact damage work is O(parts) per hit; it does not allocate.
- The damage model is not thread-safe. Run it on the simulation thread, after

the SDF collision step, before fracture.

Tests:

```
ctest --test-dir build-dev -R "sdf_(physics|damage|destructible|fracture|debris)" --
output-on-failure
```

- `sdf_physics`: SDF sampling, contacts, manifolds and the new pre-correction

impact fields.

- `sdf_damage`: material clamp, parts, connections, point lookup, default rule,

custom policy, thickness and break propagation.

- `sdf_destructible`, `sdf_fracture`, `sdf_debris`: the geometry and fragment

primitives the damage model feeds.

10. API quick reference

Type or function	Purpose
`SdfContact`	Resolved SDF contact with impact fields
`SdfMaterialProfile`	Per-material physical data
`SdfPart`	Modular piece with integrity and material id
`SdfConnection`	Structural link between two parts
`SdfImpact`	Resolved hit ready for a damage rule
`SdfImpactResult`	Damage, break and connection result
`SdfThicknessInward`	Local material depth along the normal
`MakeSdfImpact`	Contact to impact conversion
`SdfDamageModel`	Fixed-capacity part/connection store
`ApplyConnectionLoad`	Adds structural load and breaks at `Strength`
`SdfDefaultDamage`	Neutral default rule
`SdfDamagePolicy`	Custom rule callback
`PhysicsSystem::SetFixedRotation`	Locks or releases a body's angular freedom
`PhysicsSystem::SetBullet`	Enables or disables continuous collision detection
`TransportGroupSystem`	Creates, nests, carries, detaches and refreshes transport groups
`TransportGroupPanel`	Editor authoring panel for group roots, members and policies

11. Transport groups: elevators, trains and platforms

A transport group is a kinematic carrier, not a rigid-body hierarchy. It solves

the case where a platform moves under a character or a train carries crates that

must still be able to move on their own.

Do not build this from `Parent2D`: that component composes position, scale and

rotation into the world transform and is intended for visuals. It would make a dynamic crate inherit the train's rotation and scale. The transport group is a separate transform graph.

Components

Component	Fields	Meaning
`TransportGroupComponent`	`GroupId`, `ParentGroupId`, `RotationPolicy`, `Flags`, `BoundsMin`, `BoundsMax`, `ParentJoint`	Placed on the group root
`CarrierMemberComponent`	`GroupId`, `Mode`, `Flags`, `LocalPosition`, `LocalRotation`, `RelativeVelocity`, `RelativeAngularVelocity`, `Joint`	Placed on a carried entity
`TransportPath`	`Points`, `Count`, `Closed`	Optional polyline for a kinematic root
`TransportMotion`	`Speed`, `Acceleration`, `Deceleration`, `DwellSeconds`, `Distance`, `Direction`, `Axis`, `Flags`	Kinematic path or linear motion; the ramp fields are reserved for a controller
`TransportSurface`	`Friction`, `TangentSpeed`, `OneWay`, `AllowJump`	Standing/conveyor surface policy

All transport components are registered in the component registry and are visible in the inspector. Group ids are stable integers, not EnTT handles. `ParentJoint` and `Joint` are runtime Box2D ids: do not author them, and the system validates or recreates them after a scene load. `TransportPath` exposes `Count` and `Closed` to reflection; its point array is authored programmatically or by a project asset tool. Transport motion and surface are not part of the rollback `SnapshotSchema::BuiltIns`; a networked game must replicate or recreate them explicitly.

Member modes

Mode	Behaviour
`kCarrierModeRigid`	Exact attachment; a Box2D weld joint is used when both bodies exist, otherwise the exact local transform is applied
`kCarrierModeCarriedDynamic`	Body stays dynamic; the carrier adds its velocity and the body keeps its own relative motion
`kCarrierModeSupport`	Standing contact: normal support, friction-limited tangent velocity and a grounded runtime bit
`kCarrierModeConveyor`	Surface tangent velocity for belts and moving walkways

kCarrierFlagInheritRotation controls whether a member follows the carrier's rotation. A character should usually keep its own upright rotation; a crate can inherit it.

Motion, support and DOD

TransportMotion advances before the group delta is computed. A path root is sampled by arc length; Closed, ping-pong, stop-at-ends and face-direction are flags. A root without a path moves along Axis. A root with RigidBody2D is teleported so the solver sees the pose; a transform-only root is moved directly.

Support and Conveyor use TransportSurface, not a blind velocity copy. Support preserves the member's relative motion and applies friction toward the carrier surface; Conveyor drives the tangent toward TangentSpeed. The geometric support test reads the carrier/member AABB and stores kCarrierFlagGrounded, so the same rule works on Box2D and DOD.

DOD has no joints, weld or CCD. TransportGroupSystem::CapabilitiesFor() reports this; Rigid falls back to local-pose teleport, while welded trains need the Box2D backend. Motion is fixed-step and distance-based, so it is deterministic for the same starting state. TransportPath, TransportMotion and TransportSurface are not part of SnapshotSchema::BuiltIns; a rollback or networked game must recreate or replicate them. Runtime joint handles and group state are transient.

Rotation policy

RotationPolicy is metadata for the game and the editor:

- kTransportRotationFixed: the root angular degree of freedom is locked through PhysicsSystem::SetFixedRotation; the root may still translate and may be a dynamic body.
- kTransportRotationAuthored: the game/animation path owns the root rotation; fixed rotation is released.

- `kTransportRotationPhysics`: free physics rotation, including torque and collisions. This is the explicit full-physics tier.

The system never adds torque to a root. Fixed locks it; Authored leaves it to the path; Physics leaves it to the solver.

Driving the system

```
Fby::TransportGroupSystem Groups;
Groups.CreateGroup(Registry, TrainRoot, 1);
Groups.Attach(Registry, Crate, 1, Fby::kCarrierModeRigid);
Groups.Attach(Registry, Character, 1, Fby::kCarrierModeSupport);

for (const Tick& ScopedTick : Ticks)
{
    Groups.PrePhysics(Registry, Physics, ScopedTick.Dt);
    Physics.Step(Registry, ScopedTick.Dt);
    Groups.PostPhysics(Registry, Physics, ScopedTick.Dt);
}
```

`PrePhysics` computes group deltas, applies parent movement to nested roots and injects carrier velocity into dynamic members. `PostPhysics` refreshes local poses, relative velocities and detach conditions. The order matters: the physics step must sit between the two calls.

Nested groups

A train can own a lift subgroup:

```
Groups.CreateGroup(Registry, TrainRoot, 1);
Groups.CreateGroup(Registry, LiftRoot, 2, 1);
Groups.SetParentGroup(Registry, 2, 1);
```

Nested roots inherit the parent delta once. When both roots have physics bodies, a parent weld joint carries the child with the solver; otherwise the parent delta is applied to the transform. A member attached to the lift gets the lift's combined world delta, not the train delta twice. Parent ids are validated against cycles.

Attach and detach

`Attach` stores the member's local pose and, for a dynamic body, its velocity relative to the carrier. `Detach` removes the member:

- `CarriedDynamic` keeps the current body velocity, so inertia is already

preserved;

- `Rigid` destroys its weld joint and leaves the solver velocity on the body;

a body-less rigid member keeps its current transform;

- `Detach(..., false)` zeroes the body velocity for an explicit reset.

A member can request detachment by setting `kCarrierFlagDetachRequested`. A group with `kTransportGroupAutoDetachOutside` also detaches members whose local point leaves `BoundsMin`/`BoundsMax`. The game decides the condition; the system only executes it.

Editor authoring

The editor exposes a dedicated `Transport Groups` panel from the View menu. It creates a group on the selected entity, edits the group id, parent id, rotation policy, enabled/auto-detach flags and bounds, attaches the selected entity as a member, lists members, detaches them and removes the selected group. For a selected root it can add and edit `TransportMotion` and `TransportSurface`; for a wheel or chassis it also exposes the `Wheel2D`/`Vehicle2D` drive, suspension and steering fields. The same data is still ordinary registered components, so it is visible in the generic inspector and follows the project's component persistence.

Script access

The script runtime can receive the group service with `ScriptRuntime::SetTransportGroups`. The native functions are:

Native	Arguments
`transport_attach`	Group id, mode (attaches the running entity)
`transport_detach`	No arguments (detaches the running entity)
`transport_set_parent`	Group id, parent group id
`transport_create`	Group id, parent id, rotation policy (creates a group on the running entity)

The components are also registered in the ECS bridge, so a graph can set `CarrierMemberComponent::Flags` and request a detach without a native call.

Dynamic roots and high speed

- A root may be kinematic or dynamic. `Fixed` locks only the angular degree of

freedom, so an elevator can be a dynamic body that translates but does not tip. `Physics` keeps the full solver rotation.

- A carriage body remains a normal Box2D body: collisions, gravity and damping are unchanged.
- On Box2D, carriers and carried bodies automatically enable continuous collision detection once their speed exceeds `kTransportHighSpeedPixelsPerSecond`. Fixed-step substepping is still the authoring choice for extreme speeds; the automatic path is a safety net, not a replacement for a sane fixed step.
- A rigid member that inherits rotation and has both bodies present is held by a weld joint, not by a per-step teleport. The no-inherit and body-less fallbacks use the exact local transform. This removes the teleport-vs-collision conflict for normal rigid cargo.
- Nested roots that both own bodies are carried by a parent weld joint. Nested roots without bodies use the parent delta path. Both paths are supported.

12. Vehicle wheels

`WheelSystem` is a small, explicit vehicle base built on Box2D wheel joints. It owns suspension, per-wheel drive and braking. It deliberately does not own engine RPM, gears, clutch, differential, tire slip, tire wear, or steering geometry.

Components

Component	Fields	Meaning
`Vehicle2D`	`Throttle`, `Brake`, `Steering`, `MaxMotorTorque`, `MaxSpeed`, `SteeringTorque`, `Flags`	Chassis input and limits
`Wheel2D`	`Chassis`, `LocalAnchor`, `SuspensionAxis`, `Radius`, `SuspensionHertz`, `SuspensionDamping`, `DriveWeight`, `BrakeWeight`, `Flags`, `Joint`	One wheel body

A wheel entity is a normal dynamic `RigidBody2D` with a `Collider2D`. The system

does not create or place wheel bodies automatically; that is an authored chassis

choice. `Joint` is a runtime packed Box2D wheel joint and must not be serialized.

Order

```
WheelSystem::Initialize(physics)
WheelSystem::Update(registry, physics, fixedDt)
PhysicsSystem::Step(registry, fixedDt)
```

`Update` creates missing joints lazily and sets spring, motor speed and torque

before the solver step. It does not overwrite linear velocity.

Drive, brake and steering

- `Throttle` maps to `throttle * MaxSpeed / Radius` wheel angular speed.
- Per-wheel torque is `abs(throttle) * MaxMotorTorque * DriveWeight`.
- `DriveWeight = 0` makes a free wheel; front/rear/all-wheel drive is authored

per wheel, not assumed by the engine.

- `Brake` disables the wheel motor and applies the brake torque plus explicit

angular damping, so a free-spinning wheel actually stops.

- `VehicleFlagHandbrake` brakes every wheel with a nonzero `BrakeWeight`.
- `VehicleFlagArcadeSteering` is opt-in. Only wheels with `kWheelFlagSteer`

apply the speed-scaled chassis yaw torque. Physical steering geometry is a

game/plugin decision because Box2D wheel joints do not expose a steering angle.

- The engine does not add a differential or automatic reverse gear.

Backend

DOD has no wheel joints. `WheelSystem::Initialize` returns `false` and

`Available()` reports it. Wheeled vehicles require the Box2D backend.

See [vehicle wheels](../../design/vehicle-wheels.md) for the authored/game boundary.